Universidade Estadual de Santa Cruz – UESC

Relatórios II de Implementações de Métodos da Disciplina Análise Numérica

Relatório de implementações realizadas por Thainá Guimarães Santos

Disciplina Análise Numérica.

Curso Ciência da Computação

Semestre 2025.2

Professor Gesil Sampaio Amarante II

Ilhéus – BA
2025

ÍNDICE

Linguagem Escolhida e justificativas.....5
Estratégia de Implementação.....6
Figura 1 – Cálculo dos coeficientes e do coeficiente de correlação..... 6
Estrutura dos Arquivos de Entrada/Saída.....7
Figura 2 – Exemplo de entrada.....7
Figura 3 – Exemplo de saída.....7
Dificuldades enfrentadas..... 8
Estratégia de Implementação:.....9
Figura 4 – Construção da matriz de Vandermonde e cálculo dos coeficientes..... 9
Estrutura dos Arquivos de Entrada/Saída.....9
Figura 5 – Exemplo de entrada.....9
Figura 6 – Exemplo de saída.....10
Dificuldades enfrentadas..... 10
Estratégia de Implementação:.....11
Figura 7 – Construção do sistema para aproximação contínua.....11
Estrutura dos Arquivos de Entrada/Saída.....12
Figura 8 – Exemplo de entrada.....12
Figura 9 – Exemplo de saída.....12
Dificuldades enfrentadas..... 13
Figura 10: Entrada para os métodos de regressão linear e aproximação polinomial discreta..... 15
Figura 11: Saída com o método de regressão linear..... 15
Figura 12: Saída com o método de aproximação polinomial discreta.....16
Figura 13: Entrada para o método de aproximação polinomial contínua.... 16
Figura 14: Saída para o método de aproximação polinomial contínua.....17
Estratégia de Implementação..... 18
Figura 15: Trecho do código responsável pelo cálculo do polinômio de Lagrange.. 18
Estrutura dos Arquivos de Entrada e Saída.....19
Figura 16: Exemplo de arquivo entrada.txt para o método de Lagrange..... 19
Figura 17: Exemplo de arquivo saida_final.txt para o método de Lagrange..... 20
Dificuldades Enfrentadas..... 20
Estratégia de Implementação..... 21
Figura 18: Trecho do código responsável pelo cálculo do polinômio por diferenças divididas de Newton..... 21
Estrutura dos Arquivos de Entrada e Saída.....22
Figura 19: Exemplo de arquivo entrada.txt para o método de Newton.....22
Figura 20: Exemplo de arquivo saida_final.txt para o método de Newton.....23
Dificuldades Enfrentadas..... 23
Figura 21: Entrada para os métodos Polinômios de Lagrange e Polinômios por Diferenças Divididas de Newton..... 24

Figura 22: Saída para o método Polinômios de Lagrange..... 24
Figura 23: Saída para o método Polinômios por Diferenças Divididas de Newton.. 25
Estratégia de Implementação..... 26
Figura 24: Código da função para derivada numérica de primeira ordem.....26
Estrutura dos Arquivos de Entrada/Saída.....26
Figura 25: Exemplo de arquivo entrada.txt para a derivada de primeira ordem..... 27
Figura 26: Exemplo de arquivo saida.txt para a derivada de segunda ordem.....27
Dificuldades Enfrentadas:.....28
Estratégia de Implementação..... 29
Figura 27: Trecho de código da função derivada_segunda_ordem..... 29
Estrutura dos Arquivos de Entrada e Saída.....30
Figura 28: Exemplo de entrada.txt para derivação de 2° ordem.....30
Figura 29: Exemplo de saida_final.txt gerado pelo programa.....30
Dificuldades Enfrentadas.....31
Figura 30: Entrada para o método de derivação de 1° ordem e 2° ordem.....33
Figura 31: Saída para o método de derivação de 1° ordem..... 33
Figura 32: Saída para o método de derivação de 2° ordem..... 34
Estratégia de Implementação..... 35
Figura 33 - Cálculo pelo Método dos Trapézios Simples:..... 35
Estrutura dos Arquivos de Entrada e Saída.....36
Figura 34: Exemplo de entrada.txt para Trapézios Simples.....36
Figura 35: Exemplo de saida_final.txt para Trapézios Simples..... 36
Dificuldades Enfrentadas.....37
Estratégia de Implementação..... 38
Figura 36 - Cálculo pelo Método dos Trapézios Múltiplos:..... 38
Estrutura dos Arquivos de Entrada e Saída.....39
Figura 37: Entrada.txt para Trapézios Múltiplos.....39
Figura 38: Saida_final.txt para Trapézios Múltiplos.....39
Dificuldades Enfrentadas..... 40
Estratégia de Implementação..... 41
Figura 39: Cálculo pelo Método de Simpson 1/3:..... 41
Estrutura dos Arquivos de Entrada e Saída.....42
Figura 40: Entrada.txt para Simpson 1/3.....42
Figura 41: Saida_final.txt para Simpson 1/3.....42
Dificuldades Enfrentadas.....43
Foi necessário garantir que o número de subintervalos fosse par..... 43
Outra dificuldade foi organizar corretamente os índices para aplicar multiplicadores diferentes para cada ponto, evitando erros na soma ponderada..... 43
Estratégia de Implementação:..... 44
Figura 42: Cálculo pelo Método de Simpson 3/8:..... 44
Estrutura dos Arquivos de Entrada e Saída:.....45
Figura 43: Entrada.txt para Simpson 3/8.....45

Figura 44: Saida.txt para Simpson ⅜.....45
Dificuldades enfrentadas:.....46
Estratégia de Implementação:..... 47
Figura 45: Trecho de código Extrapolação de Richardson.....47
Estrutura dos Arquivos de Entrada e Saída:.....48
Figura 46: Entrada.txt para Extrapolação de Richardson..... 48
Figura 47: Saida.txt para Extrapolação de Richardson..... 48
Dificuldades enfrentadas:.....49
Estratégia de Implementação:..... 50
Figura 48: Entrada.txt para Quadratura de Gauss.....50
Estrutura dos Arquivos de Entrada e Saída:.....51
Figura 49: Entrada.txt para Quadratura de Gauss.....51
Figura 50: Saida.txt para Quadratura de Gauss.....51
Dificuldades enfrentadas:.....52
Figura 51: Entrada para os métodos de Integração numérica.....53
Figura 52: Saída do método de trapézios simples.....54
Figura 53: Saída do método de trapézios múltiplos.....54
Figura 54: Saída do método de Simpson ⅓..... 55
Figura 55: Saída do método de Simpson ⅜..... 55
Figura 56: Saída do método de Extrapolação de richards.....56
Figura 57: Saída do método Quadratura de Gauss com 10 pontos..... 56

Linguagem Escolhida e justificativas

Escolhi o **Python** para implementar os métodos por ser uma linguagem moderna, de sintaxe clara e intuitiva, o que facilita a escrita, leitura e manutenção do código. Sua popularidade e comunidade ativa oferecem suporte, exemplos e soluções para problemas, tornando o desenvolvimento mais ágil e seguro. Além disso, Python é portável, flexível e permite organizar o código em módulos e funções, facilitando futuras alterações. Apesar de não ser a linguagem mais rápida, sua clareza e praticidade tornam-no ideal para este projeto.

Regressão Linear

Estratégia de Implementação

O método de regressão linear busca ajustar uma reta y=a0+a1x aos dados experimentais, minimizando a soma dos quadrados dos erros.

- Os pares (x,y) são processados para calcular somatórios de x, y, x^2 e x · y.
- Os coeficientes a0 e a1 são obtidos usando fórmulas clássicas do **método dos mínimos quadrados**.
- O coeficiente de correlação r é calculado para avaliar a qualidade do ajuste.

Figura 1 – Cálculo dos coeficientes e do coeficiente de correlação

```
sx = np.sum(x_vals)
        sy = np.sum(y_vals)
        sx2 = np.sum(x_vals ** 2)
        sxy = np.sum(x_vals * y_vals)

        a1 = (n * sxy - sx * sy) / (n * sx2 - sx ** 2)
        a0 = (sy - a1 * sx) / n

        # Coeficiente de correlação
        r_num = n * sxy - sx * sy
        r_den = np.sqrt((n * sx2 - sx ** 2) * (n * np.sum(y_vals ** 2) - sy ** 2))
        r = r_num / r_den if r_den != 0 else 0
```

Estrutura dos Arquivos de Entrada/Saída

Entrada (entrada1.txt): cada linha contém pares (x,y)separados por espaço.

Figura 2 – Exemplo de entrada

```
1,2 2,3 3,5
2,4 3,6 4,8
```

Saída (saida_final.txt): cada linha contém a equação da reta e o coeficiente de correlação.

Figura 3 – Exemplo de saída

```
[Método: Regressão Linear]
y = 3.3333e-01 + 1.5000e+00x | r = 0.9820
y = 0.0000e+00 + 2.0000e+00x | r = 1.0000
```

Dificuldades enfrentadas

Durante o desenvolvimento da regressão linear, uma das principais dificuldades encontradas foi assegurar o cálculo exato dos coeficientes, já que o processo envolve diversas operações com somatórios e manipulação de vetores. Foi preciso atenção redobrada para evitar inconsistências, como erros de indexação ou incompatibilidades no tamanho dos vetores utilizados nas operações.

Outro desafio relevante esteve relacionado à estabilidade numérica dos resultados. Em situações nas quais os valores de entrada possuíam pouca variação ou apresentavam magnitudes muito elevadas, tornava-se necessário tomar cuidado para evitar divisões por números muito pequenos, o que poderia comprometer a precisão da regressão.

Por fim, o tratamento dos dados de entrada também exigiu cautela. A leitura e interpretação dos pares ordenados precisavam ser feitas de forma rigorosa, garantindo que o formato estivesse correto e que cada conjunto de pontos fosse processado adequadamente para o cálculo final da reta de regressão.

Aproximação Polinomial Discreta

Estratégia de Implementação:

Aproximação por polinômio discreto utilizando mínimos quadrados:

1. Construção da matriz de Vandermonde a partir dos valores xi.
2. Resolução do sistema linear via np.linalg.lstsq para obter os coeficientes do polinômio.
3. Polinômio resultante é formatado para análise.

Figura 4 – Construção da matriz de Vandermonde e cálculo dos coeficientes

```
matriz_vand = np.vander(x_vals, grau + 1, increasing=False)
    coef = np.linalg.lstsq(matriz_vand, y_vals, rcond=None)[0]

    expr = " + ".join([f"{c:.4e}x^{grau - i}" for i, c in enumerate(coef)])
```

Estrutura dos Arquivos de Entrada/Saída

Entrada (entrada1.txt): linhas com pares (x,y), e o grau do polinômio informado separadamente.

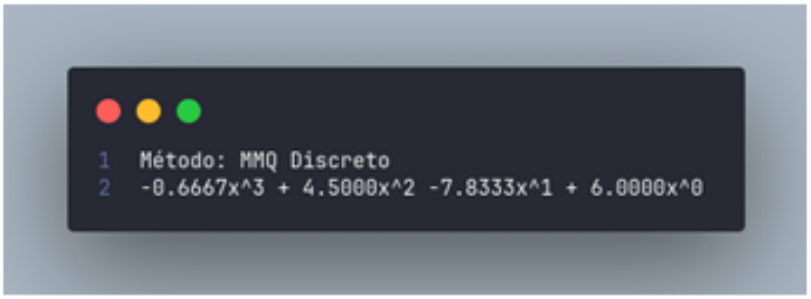
Figura 5 – Exemplo de entrada

```
1,2 2,3 3,5 4,4
3
```

=

Saída (saida_final.txt): polinômio ajustado por linha de dados.

Figura 6 – Exemplo de saída



Dificuldades enfrentadas

No método dos Mínimos Quadrados Discretos (MMQ Discreto), uma das maiores dificuldades foi estruturar e manipular corretamente os dados de entrada para formar o sistema linear associado às potências dos valores de x. A montagem da matriz de Vandermonde exigiu bastante atenção, uma vez que qualquer erro na disposição dos termos poderia comprometer o cálculo dos coeficientes do polinômio ajustado.

Outro ponto crítico foi manter a estabilidade numérica durante a resolução do sistema, principalmente em casos de polinômios de grau elevado. Nesses cenários, os coeficientes tendem a assumir valores muito grandes ou extremamente pequenos, o que pode causar perda de precisão e dificultar a convergência do método.

Também foi importante garantir que os dados de entrada estivessem bem estruturados, com pares (x, y) devidamente organizados e no formato esperado. Qualquer inconsistência na leitura ou formatação dos dados poderia gerar resultados incorretos ou até inviabilizar a execução do algoritmo. Dessa forma, o foco esteve em assegurar que o método produzisse um polinômio representativo e estável, capaz de ajustar adequadamente os pontos fornecidos.

Aproximação Polinomial Contínua

Estratégia de Implementação:

O método ajusta um polinômio a uma função contínua f(x) no intervalo [a,b] usando mínimos quadrados:

1. A função é passada como string e avaliada com eval em um contexto seguro.

2. Integrais necessárias para calcular os coeficientes são aproximadas pelo método do trapézio.

3. Um sistema linear é resolvido para determinar os coeficientes do polinômio.

Figura 7 – Construção do sistema para aproximação contínua

```python
x_vals = np.linspace(limite_a, limite_b, n_pontos)

    mat_A = np.zeros((grau + 1, grau + 1))
    vet_B = np.zeros(grau + 1)

    for i in range(grau + 1):
        for j in range(grau + 1):
            mat_A[i, j] = np.trapz(x_vals ** (i + j), x_vals)
        vet_B[i] = np.trapz(f(x_vals) * x_vals ** i, x_vals)

    coef = np.linalg.solve(mat_A, vet_B)[::-1]
    resultado = " + ".join([f"{c:.4e}x^{grau - i}" for i, c in enumerate(coef)])
    return f"f(x) ≈ {resultado}"
```

Estrutura dos Arquivos de Entrada/Saída

Entrada (entrada2.txt): cada linha contém função e intervalo separados por ;.

Figura 8 – Exemplo de entrada

```
x**2 + 3*x;0,5
np.sin(x);0,3.1416
```

Saída (saida_final.txt): polinômio ajustado correspondente à função.

Figura 9 – Exemplo de saída

```
[Método: Ajuste Contínuo]
f(x) ≈ -6.4936e-14x^3 + 1.0000e+00x^2 + 3.0000e+00x^1 + 5.3842e-13x^0
f(x) ≈ 5.3284e-07x^3 + -4.1770e-01x^2 + 1.3122e+00x^1 + -5.0465e-02x^0
```

Dificuldades enfrentadas

Na implementação do método de Aproximação Polinomial Contínua, a principal dificuldade esteve na obtenção precisa das integrais utilizadas para compor o sistema de equações lineares. Como essas integrais envolvem a multiplicação da função original por diferentes potências de x, foi necessário aplicar métodos numéricos adequados para alcançar um equilíbrio entre precisão e eficiência computacional.

Outro desafio significativo foi a resolução do sistema linear resultante, especialmente para polinômios de grau elevado, nos quais a matriz associada tende a apresentar problemas de condicionamento numérico. Isso exigiu um cuidado especial na escolha das técnicas de solução e no controle de erros de arredondamento.

Além disso, foi preciso atenção na etapa de avaliação da função e no tratamento dos limites do intervalo de integração, garantindo que o método operasse corretamente mesmo para funções mais complexas ou intervalos assimétricos. O objetivo foi manter a fidelidade matemática do processo, assegurando que a aproximação polinomial refletisse de forma estável o comportamento contínuo da função original.

Problemas estabelecidos

O programa desenvolvido foi projetado para aceitar múltiplas funções como entrada, possibilitando o processamento automatizado de diferentes problemas de interpolação e aproximação de funções. Cada função inserida corresponde diretamente a uma das questões propostas na lista — especificamente, 8.1 (a), 8.1 (b), 8.5, 8.11 (a), 8.11 (b), 10.2, 10.6 e 10.9. Dessa forma, o código permite executar todos os cálculos de forma sequencial e estruturada, garantindo eficiência e clareza na resolução de cada caso. Além disso, o formato adotado favorece a organização dos resultados, assegurando que cada saída esteja associada corretamente à sua respectiva questão.

Figura 10: Entrada para os métodos de regressão linear e aproximação polinomial discreta

```
1980,8300 1985,9900 1990,10400 1993,13200 1994,13600 1996,13700 1998,14600
1980,1688 1985,1577 1990,1397 1993,1439 1994,1418 1996,1385 1998,1415
0.1,0.62 0.2,0.63 0.3,0.64 0.4,0.66 0.5,0.68 0.6,0.71 0.7,0.76 0.8,0.81 0.9,0.89 1,1
1980,248.8 1985,398 1989,503.7 1992,684.9 1994,749.9 1995,793.5 1997,865.7
1980,355.3 1985,438 1989,487.7 1992,617.8 1994,658.1 1995,674.6 1997,707.6
0,0 10,6 20,4
500,2.74 1000,5.48 1500,7.90 2000,11.00 2500,13.93 3000,16.43 3500,20.24 4000,23.52
0.0,0.0 0.5,0.0125 1.0,0.06 1.5,0.195 2.0,0.5 2.5,1.0875 3.0,2.1 3.5,3.71 4.0,6.12 4.5,9.5625
```

Figura 11: Saída com o método de regressão linear

```
Método: Regressão Linear

362.48541*x + -709699.53326
-16.29872*x + 33922.55893
0.38909*x + 0.52600
37.38066*x + -73791.84453
22.04837*x + -43319.83203
0.20000*x + 1.33333
0.00590*x + -0.61893
1.85900*x + -1.84800
```

Figura 12: Saída com o método de aproximação polinomial discreta

```
Método: MMQ Discreto (Grau 2)

6.45743*x**2 + 25324.36904*x - 24834825.77445
1.15007*x**2 + 4591.11971*x - 4583394.77095
0.56439*x**2 + 0.23174*x - 0.65017
0.62554*x**2 + 2450.43701*x - 2399726.04790
0.39518*x**2 + 1549.60095*x - 1519295.76902
-0.04000*x**2 - 1.00000*x
0.00000*x**2 - 0.00462*x - 0.44357
0.79750*x**2 + 1.72975*x - 0.54450
```

Para a aplicação do método de aproximação polinomial contínua, foi utilizado o problema 11.1.

Figura 13: Entrada para o método de aproximação polinomial contínua

```
1/(((x)**(5))*((e)**(1.432/2000)-1));0.00004,0.00007
1/(((x)**(5))*((e)**(1.432/2200)-1));0.00004,0.00007
1/(((x)**(5))*((e)**(1.432/2250)-1));0.00004,0.00007
1/(((x)**(5))*((e)**(1.432/2400)-1));0.00004,0.00007
1/(((x)**(5))*((e)**(1.432/2500)-1));0.00004,0.00007
1/(((x)**(5))*((e)**(1.432/2600)-1));0.00004,0.00007
1/(((x)**(5))*((e)**(1.432/2750)-1));0.00004,0.00007
1/(((x)**(5))*((e)**(1.432/2800)-1));0.00004,0.00007
1/(((x)**(5))*((e)**(1.432/3000)-1));0.00004,0.00007
```

Figura 14: Saída para o método de aproximação polinomial contínua.

```
Método: MMQ Continuo (Grau 3)

-37225849148904544251839296889050628096.00000*x**3 - 69358995242975954069785195157913 6.00000*x**2 + 192515380633230453616039428096.00000*x - 191352585562576489933701 12.00000
-40949766922683431801375756904082440192.00000*x**3 - 76297378142546900591940675934617 6.00000*x**2 + 211773811644387035223806705664.00000*x - 210494695433184933524275 20.00000
-41880746370144296167074465266884673536.00000*x**3 - 78031973874922076170099771624652 8.00000*x**2 + 216588419417945427508855832576.00000*x - 215280222921481132213990 4.00000
-44673684720581640879342029410484092928.00000*x**3 - 83235761087056392697624222970675 2.00000*x**2 + 231032242780277247251692126208.00000*x - 229636805427773943190650 88.00000
-46535643626754879992824662822240649216.00000*x**3 - 86704952572771252320931205978521 6.00000*x**2 + 240661458385582650069742518272.00000*x - 239207860462202971488256 00.00000
-48397602537002765802386540910182662144.00000*x**3 - 90174144066078013582961430809804 8.00000*x**2 + 250290674011960325175515283456.00000*x - 248778915517576966150553 60.00000
-51190540909068242052375693605570871296.00000*x**3 - 95377931318509804588737893498880 0.00000*x**2 + 264734497486143298973120593920.00000*x - 263135498135045220325130 24.00000
-52121520368014406997627046180119642112.00000*x**3 - 97112527072285192081995515874508 8.00000*x**2 + 269549105319099526006070837248.00000*x - 267921025682379996523397 12.00000
-55845438210671441718555959504192667648.00000*x**3 - 104050910100189099495594201081446 4.00000*x**2 + 288807536686462409166196572160.00000*x - 287063135907043231641108 48.00000
```

Polinômios de Lagrange

Estratégia de Implementação

O método de **interpolação polinomial de Lagrange** foi implementado a partir de sua formulação clássica, com o objetivo de construir um polinômio que passe exatamente por todos os pontos fornecidos.

A lógica central do método consiste em calcular, para cada ponto do conjunto, um polinômio de base que anula todos os outros pontos, mantendo o valor do ponto atual. Cada um desses polinômios é então multiplicado pelo valor correspondente de y e somado aos demais, resultando no polinômio interpolador final.

Durante a implementação, foi adotada uma abordagem simbólica, utilizando a biblioteca **NumPy**, especialmente a classe numpy.poly1d, que facilita a manipulação algébrica dos polinômios. Essa escolha permitiu gerar a expressão expandida do polinômio de forma clara, precisa e legível.

A estrutura do código foi projetada para preservar a fidelidade à formulação teórica, mantendo o processo organizado e matematicamente coerente. A expansão dos termos ocorre automaticamente durante o cálculo, garantindo que o polinômio final seja obtido na forma reduzida.

Figura 15: Trecho do código responsável pelo cálculo do polinômio de Lagrange.

```python
def lagrange(pontos):
    xs = [p[0] for p in pontos]
    ys = [p[1] for p in pontos]
    n = len(pontos)
    pol = np.zeros(n)

    for i in range(n):
        Li = np.poly1d([1.0])
        for j in range(n):
            if i != j:
                Li *= np.poly1d([1.0, -xs[j]]) / (xs[i] - xs[j])
        pol += ys[i] * Li

    termos = [f"{c:.4g}x^{i}" for i, c in enumerate(pol[::-1]) if abs(c) > 1e-10]
    return "P(x) ≈ " + " + ".join(termos)
```

Neste trecho, o código constrói cada polinômio de base de forma iterativa, multiplicando os fatores lineares normalizados. O resultado é somado ao polinômio final, obtendo a expressão algébrica expandida.

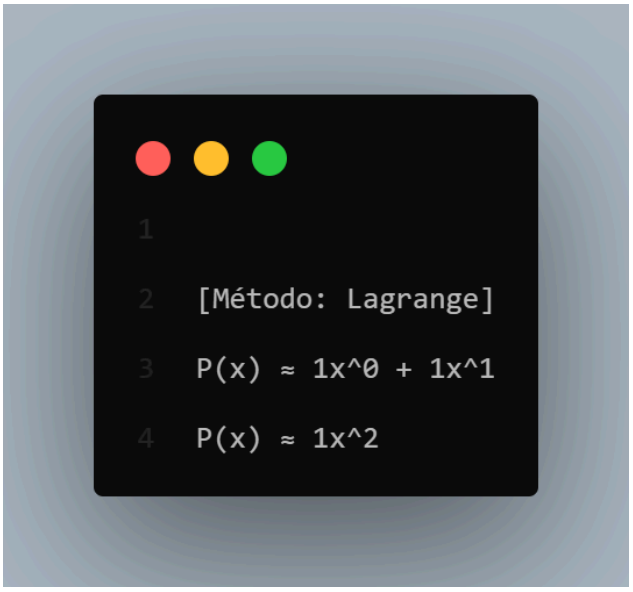
Estrutura dos Arquivos de Entrada e Saída

Arquivo de Entrada — O arquivo de entrada contém os pontos utilizados na interpolação. Cada linha representa um conjunto de pares ordenados (x,y), separados por espaço, e cada par é delimitado por vírgula.

Figura 16: Exemplo de arquivo entrada.txt para o método de Lagrange.

Arquivo de Saída — O arquivo de saída apresenta os resultados obtidos para cada conjunto de pontos. A primeira linha identifica o método utilizado, e as seguintes contêm o polinômio interpolador resultante, já expandido e formatado.

Figura 17: Exemplo de arquivo saida_final.txt para o método de Lagrange.



Dificuldades Enfrentadas

Durante a implementação, uma das maiores dificuldades foi converter a formulação matemática em uma estrutura computacional eficiente e confiável.
A multiplicação sucessiva dos termos lineares e o controle das divisões exigiram atenção redobrada para evitar erros de índice e perda de precisão numérica.

Outro desafio foi garantir que o polinômio final fosse expandido corretamente, resultando em uma expressão legível e precisa.
A biblioteca **NumPy** foi fundamental nesse processo, pois simplificou as operações entre polinômios e garantiu a exatidão dos coeficientes.

Além disso, houve a preocupação com a formatação da saída, de modo a exibir apenas os coeficientes relevantes, evitando ruídos numéricos e facilitando a leitura dos resultados.

Polinômios por Diferenças Divididas de Newton

Estratégia de Implementação

A implementação do **método de Newton por Diferenças Divididas** seguiu a estrutura clássica da técnica, baseada na construção progressiva do polinômio interpolador. Primeiro, foi criada uma **tabela de diferenças divididas** para calcular os coeficientes do polinômio. A partir desses coeficientes, o polinômio é construído de forma incremental, adicionando um termo por vez, de acordo com os pontos fornecidos.

Essa abordagem permite reutilizar cálculos anteriores, o que torna o método mais eficiente em situações onde novos pontos são adicionados. Além disso, a estrutura simbólica do código garante que o polinômio seja gerado de forma precisa e organizada.

Durante o desenvolvimento, o foco foi manter o código legível, modular e matematicamente consistente, reproduzindo fielmente a formulação teórica do método.

Figura 18: Trecho do código responsável pelo cálculo do polinômio por diferenças divididas de Newton.

```python
def newton(pontos):
    n = len(pontos)
    xs = [p[0] for p in pontos]
    ys = [p[1] for p in pontos]

    coef = ys.copy()
    for i in range(1, n):
        for j in range(n - 1, i - 1, -1):
            coef[j] = (coef[j] - coef[j - 1]) / (xs[j] - xs[j - i])

    pol = np.poly1d([0.0])
    termo = np.poly1d([1.0])
    for i in range(n):
        pol += coef[i] * termo
        termo *= np.poly1d([1.0, -xs[i]])

    c = pol.coeffs
    termos = [f"{v:.4g}x^{i}" for i, v in enumerate(c[::-1]) if abs(v) > 1e-10]
    return "P(x) ≈ " + " + ".join(termos)
```

O trecho acima implementa a construção da tabela de diferenças e o acúmulo progressivo dos termos do polinômio interpolador, garantindo que o resultado final seja simbólico e preciso.

Estrutura dos Arquivos de Entrada e Saída

Arquivo de Entrada — O formato de entrada segue o mesmo padrão adotado no método de Lagrange, com pontos representados como pares (x, y) separados por espaços e vírgulas.

Figura 19: Exemplo de arquivo entrada.txt para o método de Newton.

Nesse exemplo:

- A **primeira linha** contém três pontos: (1, 2), (2, 3) e (3, 5);
- A **segunda linha** contém quatro pontos: (0, 1), (1, 0), (2, 1) e (3, 10).

Arquivo de Saída — A saída contém os resultados gerados para cada conjunto de pontos processado, identificando o método utilizado.

Figura 20: Exemplo de arquivo saida_final.txt para o método de Newton.

```
[Método: Newton]
P(x) ≈ 2x^0 + -0.5x^1 + 0.5x^2
P(x) ≈ 1x^0 + -2x^2 + 1x^3
```

Dificuldades Enfrentadas

A principal dificuldade durante a implementação foi a construção correta da **tabela de diferenças divididas**, etapa essencial para determinar os coeficientes do polinômio. Como essa tabela depende de cálculos sucessivos e atualizações sobre os mesmos dados, qualquer erro de índice poderia comprometer o resultado final.

Outro ponto desafiador foi a montagem dinâmica do polinômio, que exige o acúmulo progressivo dos termos na forma simbólica. O uso da biblioteca **NumPy** foi fundamental para garantir a precisão e a correta expansão dos termos.

Por fim, foi preciso cuidar da **formatação dos coeficientes**, especialmente em casos com valores muito grandes ou muito pequenos, onde a notação científica foi usada automaticamente para manter a clareza dos resultados.

Problemas estabelecidos

Conforme mencionado anteriormente, o código desenvolvido foi projetado para receber múltiplas funções como entrada. Para os métodos de Polinômios de Lagrange e Polinômios por Diferenças Divididas de Newton, utilizaremos os mesmos exercícios já utilizados anteriormente. Assim, cada conjunto de entrada e saída fornecido corresponde, respectivamente, a uma das questões da lista de exercícios: 8.1 (a), 8.1 (b), 8.5, 8.11 (a), 8.11 (b), 10.2, 10.6 e 10.9.

Figura 21: Entrada para os métodos Polinômios de Lagrange e Polinômios por Diferenças Divididas de Newton

```
1980,8300 1985,9900 1990,10400 1993,13200 1994,13600 1996,13700 1998,14600
1980,1688 1985,1577 1990,1397 1993,1439 1994,1418 1996,1385 1998,1415
0.1,0.62 0.2,0.63 0.3,0.64 0.4,0.66 0.5,0.68 0.6,0.71 0.7,0.76 0.8,0.81 0.9,0.89 1,1
1980,248.8 1985,398 1989,503.7 1992,684.9 1994,749.9 1995,793.5 1997,865.7
1980,355.3 1985,438 1989,487.7 1992,617.8 1994,658.1 1995,674.6 1997,707.6
0,0 10,6 20,4
500,2.74 1000,5.48 1500,7.90 2000,11.00 2500,13.93 3000,16.43 3500,20.24 4000,23.52
0.0,0.0 0.5,0.0125 1.0,0.06 1.5,0.195 2.0,0.5 2.5,1.0875 3.0,2.1 3.5,3.71 4.0,6.12 4.5,9.5625
```

Figura 22: Saída para o método Polinômios de Lagrange

```
[Método: Lagrange]
P(x) ≈ -4.957e+17x^0 + 1.49e+15x^1 + -1.865e+12x^2 + 1.246e+09x^3 + -4.679e+05x^4 + 93.75x^5 + -0.007826x^6
P(x) ≈ -1.974e+17x^0 + 5.948e+14x^1 + -7.468e+11x^2 + 5.001e+08x^3 + -1.883e+05x^4 + 37.84x^5 + -0.003167x^6
P(x) ≈ 0.96x^0 + -10.54x^1 + 131.2x^2 + -867.4x^3 + 3405x^4 + -8280x^5 + 1.257e+04x^6 + -1.155e+04x^7 + 5878x^8 + -1268x^9
P(x) ≈ -2.822e+17x^0 + 8.507e+14x^1 + -1.069e+12x^2 + 7.16e+08x^3 + -2.698e+05x^4 + 54.23x^5 + -0.004542x^6
P(x) ≈ -1.238e+17x^0 + 3.731e+14x^1 + -4.685e+11x^2 + 3.138e+08x^3 + -1.182e+05x^4 + 23.76x^5 + -0.001989x^6
P(x) ≈ 1x^1 + -0.04x^2
P(x) ≈ -0.68x^0 + 0.003098x^1 + 1.909e-05x^2 + -3.43e-08x^3
P(x) ≈ 0.01x^1 + 0.02x^2 + 0.01x^3 + 0.02x^4
```

Figura 23: Saída para o método Polinômios por Diferenças Divididas de Newton

```
[Método: Newton]
P(x) ≈ -4.957e+17x^0 + 1.49e+15x^1 + -1.865e+12x^2 + 1.246e+09x^3 + -4.679e+05x^4 + 93.75x^5 + -0.007826x^6
P(x) ≈ -1.974e+17x^0 + 5.948e+14x^1 + -7.468e+11x^2 + 5.001e+08x^3 + -1.883e+05x^4 + 37.84x^5 + -0.003167x^6
P(x) ≈ 0.96x^0 + -10.54x^1 + 131.2x^2 + -867.4x^3 + 3405x^4 + -8280x^5 + 1.257e+04x^6 + -1.155e+04x^7 + 5878x^8 + -1268x^9
P(x) ≈ -2.822e+17x^0 + 8.507e+14x^1 + -1.069e+12x^2 + 7.16e+08x^3 + -2.698e+05x^4 + 54.23x^5 + -0.004542x^6
P(x) ≈ -1.238e+17x^0 + 3.731e+14x^1 + -4.685e+11x^2 + 3.138e+08x^3 + -1.182e+05x^4 + 23.76x^5 + -0.001989x^6
P(x) ≈ 1x^1 + -0.04x^2
P(x) ≈ -0.68x^0 + 0.003098x^1 + 1.909e-05x^2 + -3.43e-08x^3
P(x) ≈ 0.01x^1 + 0.02x^2 + 0.01x^3 + 0.02x^4
```

Derivação numérica de 1° ordem

Estratégia de Implementação

A derivação numérica de primeira ordem foi implementada utilizando a **fórmula da diferença central**, que fornece uma estimativa mais precisa da taxa de variação de uma função em torno de um ponto, sem a necessidade de calcular a derivada analítica.

A técnica considera dois pontos próximos ao valor de interesse — um à esquerda e outro à direita — avaliando a função nesses pontos para obter uma aproximação da inclinação local. O incremento h foi definido como 10−^5, garantindo boa precisão sem introduzir instabilidade numérica.

Durante a implementação, o programa foi projetado para receber **expressões algébricas** em formato de texto, avaliadas dinamicamente através da função avaliar(). Essa função utiliza um dicionário restrito de operações matemáticas seguras da biblioteca **NumPy**, permitindo o uso de funções como sin, cos, exp, log, entre outras.

Figura 24: Código da função para derivada numérica de primeira ordem.

```python
def derivada_primeira_ordem(expressao, x):
    return (avaliar(expressao, x + h) - avaliar(expressao, x - h)) / (2 * h)
```

Estrutura dos Arquivos de Entrada/Saída

Arquivo de Entrada (entrada.txt): O arquivo de entrada contém as expressões e pontos nos quais a derivada deve ser calculada. Cada linha apresenta a expressão e o valor de x separados por espaço.

Figura 25: Exemplo de arquivo entrada.txt para a derivada de primeira ordem.

```
x**2 ; 2
sin(x) ;  1.57
exp(x) ;  0
```

Arquivo de Saída (saida.txt): O arquivo de saída contém os resultados calculados para cada expressão informada. Cada linha apresenta a expressão, o ponto avaliado e o valor aproximado da derivada.

Figura 26: Exemplo de arquivo saida.txt para a derivada de segunda ordem.

```
[Método: Primeira Ordem]
Função 1: 4
Função 2: 0.000796327
Função 3: 1
```

Dificuldades Enfrentadas:

A principal dificuldade na implementação foi o controle do erro numérico ao lidar com valores muito pequenos de h. Em alguns casos, variações mínimas podem provocar grandes diferenças no resultado devido à subtração de números muito próximos.

Outro ponto de atenção foi manter a legibilidade do código e a clareza dos resultados gerados. A estrutura foi organizada para ser reutilizável, permitindo testar diferentes expressões com facilidade e garantindo coerência entre as duas implementações de derivadas.

Derivação Numérica de 2° Ordem

Estratégia de Implementação

A derivada numérica de segunda ordem foi implementada com base na **fórmula da diferença central**, uma das técnicas mais utilizadas para aproximar a concavidade de uma função em um ponto específico. Essa abordagem faz uso de três avaliações da função: no ponto de interesse x, e nos pontos x+h e x−h, o que permite estimar o comportamento local da curvatura da função sem recorrer à derivada analítica.

Durante o desenvolvimento, a escolha do **passo h=10−5^** foi feita de modo a equilibrar precisão e estabilidade numérica. Valores muito grandes de hhh diminuem a precisão da aproximação, enquanto valores muito pequenos aumentam o erro de arredondamento, devido à subtração de números muito próximos.

A implementação foi construída para aceitar **qualquer expressão algébrica** informada como texto pelo usuário, interpretando-a dinamicamente com o uso da função eval, mas de forma segura, com acesso restrito apenas a funções matemáticas necessárias. Isso garante flexibilidade sem comprometer a segurança.

Figura 27: Trecho de código da função derivada_segunda_ordem.

```
ef derivada_segunda_ordem(expressao, x):
    return (avaliar(expressao, x + h) - 2 * avaliar(expressao, x) + avaliar(expressao, x - h)) / (h ** 2)
```

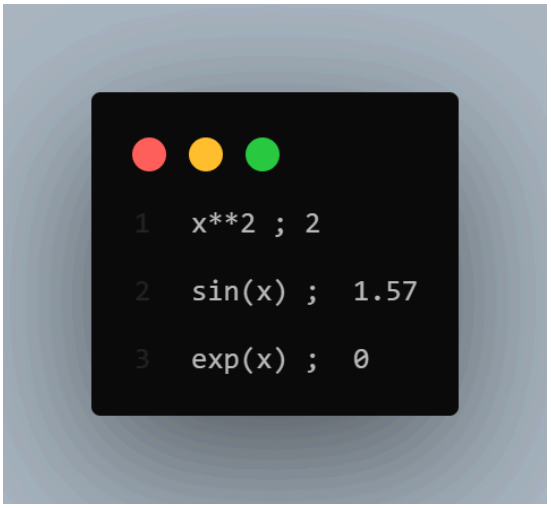
O restante da estrutura do programa foi projetado para garantir modularidade e clareza, com funções separadas para leitura de dados (carregar_dados), salvamento de resultados (salvar_resultado) e controle da execução principal (main).

A execução ocorre de forma interativa, onde o usuário escolhe o método desejado — no caso, a **Derivada Numérica de Segunda Ordem** — e os resultados são automaticamente gravados em um arquivo de saída.

Estrutura dos Arquivos de Entrada e Saída

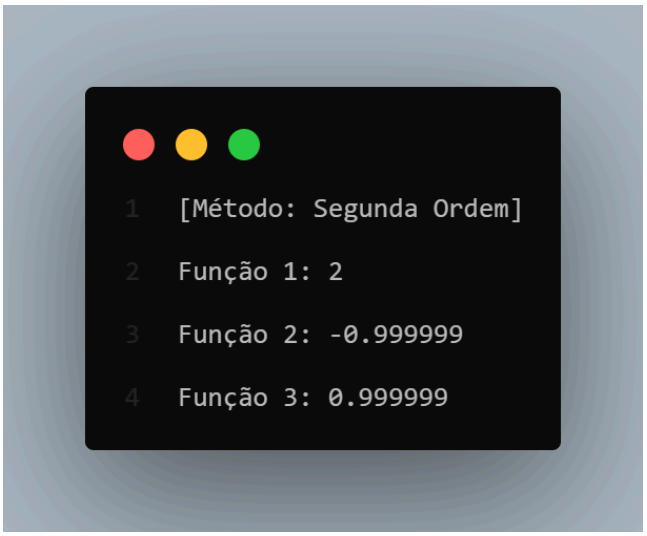
Arquivo de Entrada (entrada.txt) – O arquivo de entrada contém as expressões matemáticas e os pontos onde a derivada será avaliada.
Cada linha deve conter a função e o ponto x separados por ponto e vírgula (;) ou espaço, sendo aceitos ambos os formatos.O programa também trata erros de formatação, ignorando linhas inválidas e emitindo avisos sem interromper a execução.

Figura 28: Exemplo de `entrada.txt` para derivação de 2° ordem.



Arquivo de Saída (saida_final.txt)-- O arquivo de saída é criado automaticamente após a execução do programa. Nele, o método utilizado é identificado na primeira linha, e os resultados são listados de forma ordenada, indicando o valor aproximado da derivada para cada função informada.

Figura 29: Exemplo de saida_final.txt gerado pelo programa.



Dificuldades Enfrentadas

Durante o desenvolvimento da derivada numérica de segunda ordem, as principais dificuldades estiveram relacionadas à avaliação precisa das funções em três pontos distintos e à definição de um valor de passo hhh que garantisse equilíbrio entre precisão e estabilidade numérica.

Outra questão importante foi o tratamento de erros, especialmente para funções que não estão definidas em determinados pontos — por exemplo, `log(-1)` ou `1/x` com `x = 0`. Para lidar com isso, foi implementado um controle de exceções, permitindo que o programa informe o erro sem interromper a execução global.

Também foi necessário garantir que o usuário pudesse fornecer expressões diversas de forma segura. Para isso, o uso do `eval` foi restrito a um dicionário de funções matemáticas seguras (`sin`, `cos`, `exp`, `log`, `sqrt`, etc.), evitando riscos de execução indevida de código externo.

Por fim, a estruturação modular do código foi um ponto positivo, pois permitiu que o método de segunda ordem fosse implementado de forma independente, mas compatível com a lógica do método de primeira ordem. Assim, ambos compartilham a mesma estrutura de entrada e saída, simplificando o uso e a manutenção do programa.

Problemas estabelecidos

Para a aplicação dos métodos de Derivação Numérica de 1ª e 2ª ordem, foram selecionados os Exercícios 11.1, 11.6 e 11.11. Esses exercícios foram organizados de maneira que possam ser resolvidos automaticamente, com todos os dados de entrada reunidos em um único arquivo. Essa abordagem permite que o código execute todas as operações de forma integrada, eficiente e padronizada, considerando as especificidades de cada exercício e facilitando o processamento sequencial das aproximações numéricas.

Exercício 11.1:

1/(((x)(5))*((e)**(1.432/2000)-1));2**

1/(((x)(5))*((e)**(1.432/2200)-1));2**

1/(((x)(5))*((e)**(1.432/2250)-1));2**

1/(((x)(5))*((e)**(1.432/2400)-1));2**

1/(((x)(5))*((e)**(1.432/2500)-1));2**

1/(((x)(5))*((e)**(1.432/2600)-1));2**

1/(((x)(5))*((e)**(1.432/2750)-1));2**

1/(((x)(5))*((e)**(1.432/2800)-1));2**

1/(((x)(5))*((e)**(1.432/3000)-1));2**

Exercício 11.6:

(2*pi*0.00000595481384)/((x)(5)*((e)**(0.00071939549/x)-1));2**

(2*pi*0.00000595481384)/((x)(5)*((e)**(0.000239798497/x)-1));2**

Exercício 11.11:

(1/(0.005*(2*pi)(1/2)))*(e)**(-(1/2)*((x-4.991)/0.005)**(2));2**

Figura 30: Entrada para o método de derivação de 1° ordem e 2° ordem

```
1/(((x)**(5))*((e)**(1.432/2000)-1));2
 1/(((x)**(5))*((e)**(1.432/2200)-1));2
 1/(((x)**(5))*((e)**(1.432/2250)-1));2
 1/(((x)**(5))*((e)**(1.432/2400)-1));2
 1/(((x)**(5))*((e)**(1.432/2500)-1));2
 1/(((x)**(5))*((e)**(1.432/2600)-1));2
 1/(((x)**(5))*((e)**(1.432/2750)-1));2
 1/(((x)**(5))*((e)**(1.432/2800)-1));2
 1/(((x)**(5))*((e)**(1.432/3000)-1));2
(2*pi*0.00000595481384)/((x)**(5)*((e)**(0.00071939549/x)-1));2
(2*pi*0.00000595481384)/((x)**(5)*((e)**(0.000239798497/x)-1));2
(1/(0.005*(2*pi)**(1/2)))*(e)**(-(1/2)*((x-4.991)/0.005)**(2));2
```

Figura 31: Saída para o método de derivação de 1° ordem

```
[Método: Primeira Ordem]
Função 1: -109.074
Função 2: -119.985
Função 3: -122.713
Função 4: -130.897
Função 5: -136.352
Função 6: -141.808
Função 7: -149.991
Função 8: -152.719
Função 9: -163.631
Função 10: -0.00649969
Função 11: -0.019502
Função 12: 0
```

Figura 32: Saída para o método de derivação de 2° ordem

```
[Método: Segunda Ordem]
Função 1: 327.222
Função 2: 359.956
Função 3: 368.14
Função 4: 392.69
Função 5: 409.057
Função 6: 425.424
Função 7: 449.974
Função 8: 458.158
Função 9: 490.892
Função 10: 0.0162694
Função 11: 0.0485743
Função 12: 0
```

Integração por Trapézios Simples

Estratégia de Implementação

O método dos Trapézios Simples foi implementado utilizando a abordagem clássica de integração numérica, aproximando a área sob a curva com base apenas nos valores da função nos extremos do intervalo de integração.

A função é avaliada nos pontos inicial e final através de um avaliador genérico que interpreta expressões matemáticas fornecidas pelo usuário em formato textual. O cálculo é simples, rápido e eficiente, adequado para funções contínuas e suavemente variáveis.

O código foi estruturado para garantir legibilidade, reaproveitamento da função avaliadora e compatibilidade com os demais métodos implementados no mesmo programa.

Figura 33 - Cálculo pelo Método dos Trapézios Simples:

```python
def trapezio_simples(self, func, intervalo, n=None):
        a, b = intervalo
        fa = self._eval(func, a)
        fb = self._eval(func, b)
        return (b - a) * (fa + fb) / 2
```

Estrutura dos Arquivos de Entrada e Saída

Arquivo de Entrada (entrada.txt) O arquivo de entrada contém, em cada linha, a expressão da função, o intervalo de integração e o número de subdivisões (quando necessário), separados por ponto e vírgula.

Figura 34: Exemplo de entrada.txt para Trapézios Simples

```
x**2 ; 0,2 ; 1
sin(x) ; 0,3.14 ; 1
exp(x) ; 0,1 ; 1
```

Arquivo de Saída (saida_final.txt) – O arquivo de saída apresenta os resultados das integrais calculadas, com notação científica e quatro casas decimais.

Figura 35: Exemplo de saida_final.txt para Trapézios Simples

```
[Método: Trapézio simples]
Função 1: 4.000000e+00
Função 2: 2.500465e-03
Função 3: 1.859141e+00
```

Dificuldades Enfrentadas

Uma das principais dificuldades foi a **baixa precisão para funções não lineares**, pois o método considera apenas os extremos do intervalo. Funções com curvatura acentuada, como sen(x) e exponenciais, apresentaram resultados menos precisos.

Além disso, foi necessário garantir o **formato correto da entrada**, separando limites por vírgulas e mantendo expressões válidas para avaliação.

A ausência de subdivisões limita a precisão, especialmente em funções que variam muito no intervalo.

Integração por Trapézios Múltiplos

Estratégia de Implementação

O método dos Trapézios Múltiplos divide o intervalo de integração em várias partes menores e aplica o cálculo do trapézio simples em cada subintervalo, proporcionando uma aproximação mais precisa da área sob a curva.

A função é avaliada em cada ponto do intervalo, somando os valores extremos uma vez e os internos duas vezes. O resultado é multiplicado pelo tamanho do subintervalo, gerando a área total aproximada.

O código mantém clareza e reutiliza o avaliador de expressões, garantindo flexibilidade para diferentes funções.

Figura 36 - Cálculo pelo Método dos Trapézios Múltiplos:

```python
def trapezio_multiplo(self, func, intervalo, n):
        a, b = intervalo
        h = (b - a) / n
        xs = a + h * np.arange(0, n + 1)
        vals = np.array([self._eval(func, x) for x in xs])
        soma = vals[0] + vals[-1] + 2 * vals[1:-1].sum()
        return (h / 2) * soma
```

Estrutura dos Arquivos de Entrada e Saída

Arquivo de Entrada (entrada.txt): O arquivo de entrada contém, em cada linha, uma expressão da função a ser integrada, o intervalo de integração e o número de subdivisões (quando necessário), todos separados por ponto e vírgula.

Figura 37: Entrada.txt para Trapézios Múltiplos

```
x**2 ; 0,2 ; 4
sin(x) ; 0,3.14 ; 8
exp(x) ; 0,1 ; 10
```

Arquivo de Saída (saida.txt): O arquivo de saída apresenta os resultados das integrais calculadas. A primeira linha identifica o método utilizado. As demais linhas exibem os valores das integrais correspondentes às funções fornecidas na entrada, com notação científica e quatro casas decimais de precisão.

Figura 38: Saida_final.txt para Trapézios Múltiplos

```
[Método: Trapézio múltiplo]
Função 1: 2.750000e+00
Função 2: 1.974257e+00
Função 3: 1.719713e+00
```

Dificuldades Enfrentadas

Garantir a divisão correta do intervalo em subintervalos iguais foi essencial para a precisão do método.

Outra dificuldade foi organizar corretamente os índices ao somar os valores dos pontos internos, mantendo a clareza e evitando erros de lógica no laço de repetição.

Integração por Simpson 1/3

Estratégia de Implementação

O método de Simpson 1/3 aproxima a integral utilizando parábolas para estimar a área sob a curva. O intervalo de integração é dividido em subintervalos pares, garantindo que cada par de pontos defina a parábola usada na aproximação.

A função é avaliada nos pontos extremos e nos pontos intermediários, aplicando diferentes multiplicadores aos valores alternadamente. O avaliador genérico de funções garante que qualquer expressão válida em x possa ser usada.

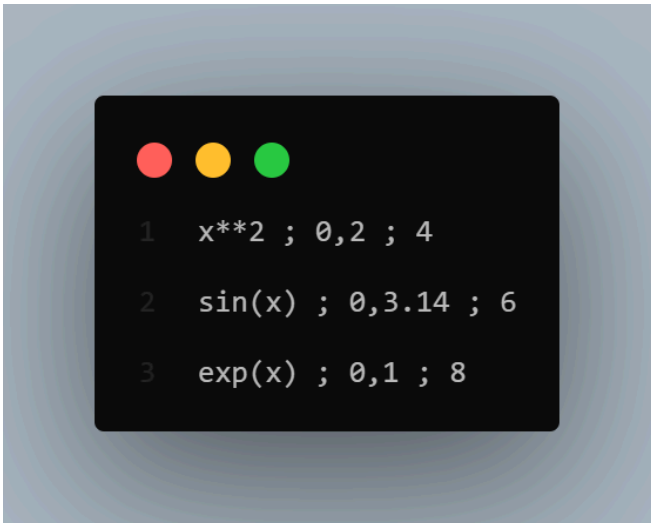
Figura 39: Cálculo pelo Método de Simpson 1/3:

```python
def simpson_1_3(self, func, intervalo, n):
        if n < 2: n = 2
        if n % 2 != 0: n += 1
        a, b = intervalo
        h = (b - a) / n
        xs = a + h * np.arange(0, n + 1)
        vals = np.array([self._eval(func, x) for x in xs])
        soma = vals[0] + vals[-1] + 4 * vals[1:-1:2].sum() + 2 * vals[2:-1:2].sum()
        return (h / 3) * soma
```

Estrutura dos Arquivos de Entrada e Saída

Arquivo de Entrada (entrada.txt): O arquivo de entrada contém, em cada linha, uma expressão da função a ser integrada, o intervalo de integração e o número de subdivisões, todos separados por ponto e vírgula.

Figura 40: Entrada.txt para Simpson 1/3



Arquivo de Saída (saida.txt): O arquivo de saída apresenta os resultados das integrais calculadas. A primeira linha identifica o método utilizado. As demais linhas exibem os valores das integrais correspondentes às funções fornecidas na entrada, com notação científica e quatro casas decimais de precisão.

Figura 41: Saida_final.txt para Simpson 1/3

Dificuldades Enfrentadas

Foi necessário garantir que o número de subintervalos fosse par.

Outra dificuldade foi organizar corretamente os índices para aplicar multiplicadores diferentes para cada ponto, evitando erros na soma ponderada.

Integração por Simpson 3/8

Estratégia de Implementação:

O método de Simpson 3/8 foi implementado com base na ideia de aproximar a área sob a curva da função utilizando subintervalos iguais dentro do intervalo de integração. Cada três subintervalos consecutivos definem uma aproximação utilizando parábolas, permitindo capturar melhor a variação da função em intervalos curvos. A implementação calcula a largura de cada subintervalo e avalia a função nos pontos necessários, somando os valores com pesos diferenciados para os pontos intermediários e finais, garantindo uma aproximação equilibrada e precisa. A função é fornecida como texto, interpretada dinamicamente pelo avaliador genérico de expressões, o que confere flexibilidade para qualquer função válida em x. A abordagem mantém a simplicidade do código e ao mesmo tempo permite generalidade, podendo ser aplicada a funções com diferentes comportamentos.

Figura 42: Cálculo pelo Método de Simpson 3/8:

```python
def simpson_3_8(self, func, intervalo, n):
        if n < 3: n = 3
        if n % 3 != 0: n += (3 - n % 3)
        a, b = intervalo
        h = (b - a) / n
        xs = a + h * np.arange(0, n + 1)
        vals = np.array([self._eval(func, x) for x in xs])
        soma = vals[0] + vals[-1]
        for i in range(1, n):
            soma += vals[i] * (3 if i % 3 != 0 else 2)
        return (3 * h / 8) * soma
```

Estrutura dos Arquivos de Entrada e Saída:

Arquivo de Entrada - O arquivo de entrada contém uma linha para cada função a ser integrada. Cada linha inclui a função, o intervalo de integração e o número de subdivisões, separados por ponto e vírgula. O intervalo é fornecido pelos limites inferior e superior, separados por vírgula. Por exemplo, um arquivo de entrada completo poderia conter:

Figura 43: Entrada.txt para Simpson 3/8

```
x**2 ; 0,2 ; 6
sin(x) ; 0,3.1416 ; 9
exp(x) ; 0,1 ; 12
cos(x) ; 0,1.5708 ; 6
log(x+1) ; 0,2 ; 6
```

Arquivo Saída: Cada linha representa uma função a ser integrada e as subdivisões do intervalo permitem controlar a precisão da aproximação. O método ajusta automaticamente o número de subdivisões para que seja múltiplo de três, se necessário. O arquivo de saída apresenta os resultados obtidos, incluindo a função original e o valor da integral calculado em notação científica, garantindo clareza e precisão na leitura. Por exemplo:

Figura 44: Saida.txt para Simpson ⅜

```
[Método: Simpson 3/8]
Função 1: 2.666667e+00
Função 2: 2.000382e+00
Função 3: 1.718283e+00
Função 4: 1.000060e+00
Função 5: 1.295601e+00
```

Dificuldades enfrentadas:

Uma das principais dificuldades na implementação do Simpson 3/8 foi garantir que o número de subdivisões fosse múltiplo de três, conforme exigido pelo método. O ajuste automático foi necessário para evitar erros de cálculo. Além disso, a atribuição correta dos pesos para cada ponto do subintervalo exigiu atenção, pois qualquer erro na soma ponderada comprometeria a precisão da integral. Outra dificuldade foi lidar com funções cuja variação é acentuada dentro do intervalo, o que demandou testes com diferentes números de subdivisões para validar a consistência dos resultados. O uso do avaliador genérico também exigiu cuidados especiais para evitar falhas na interpretação das funções fornecidas em formato de texto, garantindo que qualquer expressão válida pudesse ser processada sem interromper o cálculo.

Extrapolação de Richardson

Estratégia de Implementação:

O método da Extrapolação de Richardson foi implementado como uma técnica de refinamento de aproximação baseada no método do Trapézio Múltiplo. A ideia é calcular duas estimativas da integral: uma com um número n de subdivisões e outra com o dobro de subdivisões, e então combinar essas aproximações de forma a reduzir o erro de ordem inferior. A implementação reutiliza a função de trapézio múltiplo, garantindo consistência e simplicidade no código. Essa abordagem permite aumentar a precisão da integral sem exigir grandes incrementos no número de subdivisões, equilibrando eficiência computacional e exatidão do resultado. O avaliador genérico de expressões permite processar funções fornecidas em formato textual, oferecendo flexibilidade e aplicabilidade para diferentes tipos de funções matemáticas.

Figura 45: Trecho de código Extrapolação de Richardson

```python
def richardson(self, func, intervalo, n):
        Tn = self.trapezio_multiplo(func, intervalo, n)
        T2n = self.trapezio_multiplo(func, intervalo, 2 * n)
        return (4 * T2n - Tn) / 3
```

Estrutura dos Arquivos de Entrada e Saída:

O arquivo de entrada segue o mesmo padrão das integrais por trapézio múltiplo, contendo uma função por linha, o intervalo de integração e o número de subdivisões n, todos separados por ponto e vírgula. Um exemplo detalhado seria:

Figura 46: Entrada.txt para Extrapolação de Richardson

```
x**2 ; 0,2 ; 4
sin(x) ; 0,3.1416 ; 8
exp(x) ; 0,1 ; 10
cos(x) ; 0,1.5708 ; 6
log(x+1) ; 0,2 ; 4
```

O arquivo de saída apresenta os resultados refinados obtidos pela extrapolação, em notação científica com quatro casas decimais, por exemplo:

Figura 47: Saida.txt para Extrapolação de Richardson

```
[Método: Simpson 3/8]
Função 1: 2.666667e+00
Função 2: 2.000382e+00
Função 3: 1.718283e+00
Função 4: 1.000060e+00
Função 5: 1.295601e+00
```

Dificuldades enfrentadas:

A principal dificuldade foi compreender e aplicar corretamente a combinação das duas estimativas obtidas com diferentes subdivisões para reduzir o erro. Foi necessário garantir que tanto a integral com n quanto com 2n subdivisões fossem calculadas com a mesma função de base, evitando inconsistências. Outro desafio envolveu a validação da precisão obtida: testar diferentes funções e intervalos para verificar se o método realmente fornecia uma melhora significativa em relação ao trapézio múltiplo simples.

Quadratura de Gauss

Estratégia de Implementação:

A Quadratura de Gauss foi implementada aproveitando a transformação do intervalo original [a,b] para o intervalo padrão [-1,1], onde os pontos e pesos de Gauss são conhecidos. O método calcula os pontos transformados e avalia a função nesses pontos, multiplicando cada valor pelo peso correspondente, e em seguida ajusta pelo fator da mudança de intervalo. Essa abordagem permite obter uma alta precisão com um número relativamente pequeno de pontos, tornando-a eficiente para funções suaves ou de difícil integração analítica. O código reutiliza o avaliador genérico de expressões usado nos outros métodos, garantindo consistência e flexibilidade. A escolha do número de pontos n permite ajustar a precisão do método conforme a complexidade da função.

Figura 48: Entrada.txt para Quadratura de Gauss

```python
def gauss(self, func, intervalo, n):
        xi, wi = np.polynomial.legendre.leggauss(n)
        a, b = intervalo
        ts = 0.5 * (b - a) * xi + 0.5 * (b + a)
        vals = np.array([self._eval(func, t) for t in ts])
        return 0.5 * (b - a) * np.dot(wi, vals)
```

Estrutura dos Arquivos de Entrada e Saída:

O arquivo de entrada segue o mesmo formato das integrais anteriores, contendo uma função por linha, o intervalo de integração e o número de subdivisões, separados por ponto e vírgula. O número de pontos para a quadratura de Gauss é solicitado separadamente ao usuário. Um exemplo de arquivo de entrada detalhado seria:

Figura 49: Entrada.txt para Quadratura de Gauss

```
x**2 ; 0,2 ; 4
sin(x) ; 0,3.1416 ; 8
exp(x) ; 0,1 ; 10
cos(x) ; 0,1.5708 ; 6
log(x+1) ; 0,2 ; 4
```

O arquivo de saída apresenta os resultados calculados pela quadratura, em notação científica com quatro casas decimais:

Figura 50: Saida.txt para Quadratura de Gauss

```
[Método: Quadratura de Gauss]
Função 1: 2.666667e+00
Função 2: 1.999984e+00
Função 3: 1.718282e+00
Função 4: 1.000000e+00
Função 5: 1.295846e+00
```

Dificuldades enfrentadas:

Um dos principais desafios foi aplicar corretamente a transformação do intervalo de integração para [-1,1], garantindo que os pontos e pesos fossem ajustados corretamente. Outro ponto importante foi entender o funcionamento da função leggauss(n) do NumPy e integrar seus resultados com a avaliação da função em cada ponto. Além disso, para funções mais complexas ou com mudanças abruptas, foi necessário testar diferentes valores de n para obter precisão adequada sem comprometer a eficiência computacional. Garantir que todas as funções fossem avaliadas corretamente nos pontos transformados exigiu atenção especial para evitar erros de cálculo.

Problemas estabelecidos

Os exercícios 11.1, 11.6 e 11.11 foram empregados para validar os métodos de integração numérica. O código foi desenvolvido de modo a receber diferentes funções como entrada, possibilitando a execução automática e eficiente dos cálculos de cada questão. Todos os valores necessários foram reunidos em um único arquivo, com formato padronizado, o que facilita a leitura sequencial e o processamento integral das expressões propostas, garantindo clareza e organização na execução.

Figura 51: Entrada para os métodos de Integração numérica

```
1/(x**5 * (exp(1.432/2000) - 1));0.00004, 0.0007;1
1/(x**5 * (exp(1.432/2200) - 1));0.00004, 0.0007;1
1/(x**5 * (exp(1.432/2250) - 1));0.00004, 0.0007;1
1/(x**5 * (exp(1.432/2400) - 1));0.00004, 0.0007;1
1/(x**5 * (exp(1.432/2500) - 1));0.00004, 0.0007;1
1/(x**5 * (exp(1.432/2600) - 1));0.00004, 0.0007;1
1/(x**5 * (exp(1.432/2750) - 1));0.00004, 0.0007;1
1/(x**5 * (exp(1.432/2800) - 1));0.00004, 0.0007;1
1/(x**5 * (exp(1.432/3000) - 1));0.00004, 0.0007;1
(2 * pi * 0.00000595481384) / (x**5 * (exp(0.00071939549/x) - 1));0.0000393366, 0.00005895923;1
(2 * pi * 0.00000595481384) / (x**5 * (exp(0.000239798497/x) - 1));0.0000393366, 0.00005895923;1
(1 / (0.005 * (2 * pi)**0.5)) * exp(-0.5 * (((x - 4.991)/0.005)**2));4.991, 5;1
```

Figura 52: Saída do método de trapézios simples

```
[Método: Trapézio simples]
Função 1: 4.499308e+21
Função 2: 4.949400e+21
Função 3: 5.061923e+21
Função 4: 5.399492e+21
Função 5: 5.624538e+21
Função 6: 5.849584e+21
Função 7: 6.187153e+21
Função 8: 6.299676e+21
Função 9: 6.749768e+21
Função 10: 2.632362e+06
Função 11: 1.777352e+10
Função 12: 4.301032e-01
```

Figura 53: Saída do método de trapézios múltiplos

```
[Método: Trapézio múltiplo]
Função 1: 4.499308e+21
Função 2: 4.949400e+21
Função 3: 5.061923e+21
Função 4: 5.399492e+21
Função 5: 5.624538e+21
Função 6: 5.849584e+21
Função 7: 6.187153e+21
Função 8: 6.299676e+21
Função 9: 6.749768e+21
Função 10: 2.632362e+06
Função 11: 1.777352e+10
Função 12: 4.301032e-01
```

Figura 54: Saída do método de Simpson ⅓

```
[Método: Simpson 1/3]
Função 1: 1.499858e+21
Função 2: 1.649897e+21
Função 3: 1.687407e+21
Função 4: 1.799937e+21
Função 5: 1.874957e+21
Função 6: 1.949976e+21
Função 7: 2.062506e+21
Função 8: 2.100016e+21
Função 9: 2.250055e+21
Função 10: 1.627808e+06
Função 11: 1.900214e+10
Função 12: 4.626700e-01
```

Figura 55: Saída do método de Simpson ⅜

```
[Método: Simpson 3/8]
Função 1: 1.125131e+21
Função 2: 1.237685e+21
Função 3: 1.265823e+21
Função 4: 1.350238e+21
Função 5: 1.406515e+21
Função 6: 1.462792e+21
Função 7: 1.547207e+21
Função 8: 1.575345e+21
Função 9: 1.687899e+21
Função 10: 1.630143e+06
Função 11: 1.900416e+10
Função 12: 4.635280e-01
```

Figura 56: Saída do método de Extrapolação de richards

```
[Método: Richardson]
Função 1: 1.499858e+21
Função 2: 1.649897e+21
Função 3: 1.687407e+21
Função 4: 1.799937e+21
Função 5: 1.874957e+21
Função 6: 1.949976e+21
Função 7: 2.062506e+21
Função 8: 2.100016e+21
Função 9: 2.250055e+21
Função 10: 1.627808e+06
Função 11: 1.900214e+10
Função 12: 4.626700e-01
```

Figura 57: Saída do método Quadratura de Gauss com 10 pontos

```
Método: Quadratura de Gauss]
Função 1: 1.309168e+20
Função 2: 1.440132e+20
Função 3: 1.472872e+20
Função 4: 1.571095e+20
Função 5: 1.636577e+20
Função 6: 1.702059e+20
Função 7: 1.800282e+20
Função 8: 1.833023e+20
Função 9: 1.963986e+20
Função 10: 1.631930e+06
Função 11: 1.900536e+10
Função 12: 4.640697e-01
```

Considerações Finais

Durante o desenvolvimento deste trabalho, foram implementados, testados e analisados diversos métodos numéricos importantes para resolver problemas aproximados em engenharia e ciências exatas. A prática com esses algoritmos ajudou a entender melhor suas aplicações, vantagens e limitações em diferentes situações. A seguir, são apresentadas comparações que mostram em quais casos cada método é mais indicado.

Regressão Linear vs. Aproximação Polinomial
A regressão linear é mais indicada quando os dados seguem uma tendência aproximadamente reta, permitindo uma modelagem simples. Já as aproximações polinomiais — discretas ou contínuas — são mais adequadas para dados com comportamento não linear, pois oferecem maior flexibilidade e ajuste mais preciso.

Polinômios de Lagrange vs. Diferenças Divididas de Newton
Ambos são métodos de interpolação polinomial. O método de Lagrange é fácil de entender e usar quando há poucos pontos, mas se torna pouco prático quando novos dados são adicionados, pois é preciso recalcular todo o polinômio. O método de Newton, por sua vez, é mais eficiente nesse caso, pois permite aproveitar cálculos anteriores, sendo melhor em situações onde os dados são atualizados com frequência.

Derivação Numérica de 1ª e 2ª Ordem
Esses métodos são úteis quando não se tem a expressão da função, mas apenas valores em tabela. A derivada de primeira ordem mostra a taxa de variação, enquanto a de segunda ordem indica a concavidade da função. No entanto, ambos podem perder precisão se os pontos estiverem muito espaçados ou se houver ruídos nos dados.

Integração: Trapézios, Simpson e Gauss
O método dos trapézios é simples e funciona bem para funções quase lineares, mas perde precisão em funções mais complexas. Os métodos de Simpson (1/3 e 3/8) melhoram a precisão, pois usam aproximações quadráticas ou cúbicas, sendo mais adequados para funções suaves. Já a quadratura de Gauss oferece resultados muito precisos e eficientes, mas exige mais cuidado na implementação e na compreensão de seus fundamentos.

Extrapolação de Richardson
A extrapolação de Richardson é uma técnica usada para aumentar a precisão de métodos de derivação e integração, reduzindo erros. É vantajosa quando se quer maior exatidão sem precisar aumentar muito o número de pontos. No entanto, ela demanda mais cálculos, o que pode elevar o custo computacional.

Conclusão Geral

Cada método tem suas próprias condições ideais de uso. A escolha do mais adequado depende do tipo de função, do nível de precisão desejado e dos recursos disponíveis. A prática com todos esses métodos mostrou que não existe uma técnica melhor para tudo, mas sim métodos que funcionam melhor em diferentes contextos. Essa experiência reforçou a importância de entender tanto a teoria matemática quanto o comportamento computacional de cada abordagem.